Suhail Smart Teacher

Deployment and technical handover

Prepared October 10, 2026 · Based on repository state inspected for this handover

> **Release caveat:** The inspected branch is `main` at commit `e4d0fb0e096a72f9a50445a1f4894361e86c387c`. The working tree has uncommitted frontend changes, so this is not yet a clean, reproducible release. No release tag or live deployment was confirmed.

1. Source code and documentation

- Repository: github.com/OluwaseunSoft/Smart-Teacher-Platform
- Frontend source: `frontend/` — React, TypeScript, Vite, Tailwind CSS; installable PWA.
- Backend source: `backend/` — FastAPI, SQLAlchemy, provider integrations, and tests.
- Deployment: root `docker-compose.yml`; `frontend/Dockerfile`; `backend/Dockerfile`.
- Environment template: `backend/.env.example`. Do not commit real credentials or `.env`.
- Architecture and design: `docs/01-architecture.md`, `docs/02-data-model.md`, `docs/03-api-spec.md`, `docs/04-ai-pipeline.md`.
- Project overview / installation: root `README.md`.

The codebase has grown beyond parts of the design documents, notably in accounts, admin, tutor, planner, review, and gamification features. Treat source code as authoritative and reconcile documentation before using it as a release specification.

2. Architecture and database

The React/Vite/TypeScript PWA is served by Nginx. Nginx forwards `/api` to FastAPI. FastAPI uses SQLAlchemy and SQLite and invokes the selected language-model provider. The material pipeline extracts and chunks text, optionally embeds it, builds a curriculum, generates lessons/quizzes, and records source-grounding references. There is no separate vector database or job queue.

SQLAlchemy tables: `users`, `user_sessions`, `student_profiles`, `students`, `materials`, `chunks`, `concepts`, `lessons`, `questions`, `attempts`, `mastery`, `mastery_events`, `study_sessions`, `institutions`, `academic_years`, `subjects`, `chapters`, `topics`, `topic_chunks`, `review_schedules`, `tutor_conversations`, `tutor_messages`, `exam_dates`, `study_plans`, `study_plan_items`, `achievements`, `user_achievements`, `streaks`, `notifications`, `audit_logs`, and `ai_usage_logs`.

The backend calls SQLAlchemy `Base.metadata.create_all` at startup. Compose stores SQLite at `/data/suhail.db` in the persistent `backend-data` volume. No Alembic migrations or migration scripts are present; `create_all` does not migrate existing columns. Back up the database before schema-changing upgrades.

3. Server requirements and network

- **Operating system:** Docker Engine with Docker Compose v2; application containers are Linux-based. Native development also needs Python and Node.js.
- **CPU, RAM, storage:** No formal minimum is specified or benchmarked. Rough app-only planning floor (excluding local model inference): 2 vCPU, 4 GB RAM, 10 GB persistent storage.
- **GPU:** Not required for hosted AI or for running the web/API services. A GPU substantially improves local inference.
- **Ports:** Compose publishes frontend HTTP on 8080 and backend API on 8000. Ollama uses port 11434 inside the Compose network and is not published to the host by default. Vite development uses 5173.
- **Network:** Outbound HTTPS is needed initially for container images, package/model downloads, and hosted AI APIs if selected.
- **Dependencies:** Compose runs Nginx, FastAPI, and Ollama. SQLite is embedded. No PostgreSQL, Redis, or vector database is required. PDF text extraction uses `pypdf`.

Compose exposes the API port directly; restrict it with firewall rules if it should only be reached through a reverse proxy. The supplied setup serves HTTP only. Use a TLS reverse proxy for public deployment and PWA installation; HTTPS is required for installability on deployed hosts.

4. Installation, deployment, and startup

From PowerShell at the repository root:

```
Copy-Item backend\.env.example backend\.env
# Edit backend\.env: set a strong AUTH_SECRET_KEY and production settings.
docker compose up --build -d

# Fetch default local models into the persistent Ollama volume:
docker compose exec ollama ollama pull llama3.1
docker compose exec ollama ollama pull nomic-embed-text
```

- Application/PWA: `http://localhost:8080`
- API: `http://localhost:8000`
- Interactive API docs: `http://localhost:8000/docs`

For development without Docker, install backend requirements in a virtual environment, copy/edit `backend/.env.example`, and run `uvicorn app.main:app --reload` from `backend/`. In a second terminal, run `npm install` and `npm run dev` from `frontend/`; Vite is at `http://localhost:5173`.

The PWA caches its application shell, not authenticated API responses. Data-backed pages therefore still need network access. The Docker frontend build checks that its manifest and service-worker files are present.

5. Configuration and secrets

See `backend/.env.example` for all available options. Main settings:

Variable	Purpose / default
`LLM_PROVIDER`	Generation provider; default `ollama`, also `openai` or `anthropic`.
`EMBEDDING_PROVIDER`	Embedding provider; default `ollama`, or `openai`.

`LLM_MODEL`, `EMBEDDING_MODEL`	Optional model-name overrides.
`OLLAMA_BASE_URL`	Host-run backend default `http://localhost:11434`; Compose sets `http://ollama:11434`.
`OPENAI_API_KEY`, `ANTHROPIC_API_KEY`	Required only when the corresponding hosted provider is selected.
`DATABASE_URL`	Database URL; Compose forces `sqlite:////data/suhail.db`.
`CORS_ORIGINS`, `RETRIEVAL_TOP_K`, `MAX_UPLOAD_MB`	Allowed browser origins, retrieval context count, and upload limit.
`AUTH_SECRET_KEY`	JWT signing secret. Replace the development value before deployment.
`AUTH_EXPOSE_RESET_TOKEN`	Example defaults true for development; disable before public deployment.
`DEFAULT_ADMIN_EMAIL`, `DEFAULT_ADMIN_PASSWORD`	Optional bootstrap administrator.
`SEED_DEMO_DATA`	Defaults true; set false for a public deployment.

On a new, empty Docker SQLite volume, demo data is seeded if enabled; an existing non-empty database is left untouched. Compose currently starts Ollama and waits for it to become healthy even when using a hosted provider. The health endpoint reports application status and configured provider names; it does not check that a model is loaded or inference is functioning.

6. Status, logs, backup, and restore

```
docker compose ps
curl.exe http://localhost:8000/api/health
docker compose logs -f backend
docker compose logs -f frontend
docker compose logs -f ollama
```

For a consistent SQLite backup, stop the API during the copy:

```
docker compose stop backend
docker compose cp backend:/data/suhail.db .\suhail.db.backup
docker compose start backend
```

To restore, stop the backend, copy a verified backup to `backend:/data/suhail.db` using `docker compose cp`, then start the backend. Keep a separate backup copy. The Ollama model volume can generally be repopulated by downloading models again. **Do not run** `docker compose down -v` **unless you intend to delete both database and model volumes.**

7. AI integrations, costs, privacy, and errors

Provider	Default model(s)	Purpose	Credential
Ollama	`llama3.1`; `nomic-embed-text`	Local text generation and embeddings	None

OpenAI	`gpt-4o-mini`; `text-embedding-3-small`	Hosted generation and embeddings	`OPENAI_API_KEY`
Anthropic	`claude-sonnet-5-5`	Hosted generation; no Anthropic embedding integration	`ANTHROPIC_API_KEY` and a separate embedding provider

These are code defaults, not evidence that every model has been live-tested. Set provider/model choices in `.env`; integrations are in `backend/app/llm/`.

With a hosted provider selected, the app sends the relevant study-material text or excerpts for structuring, lesson and quiz generation, and embeddings. Short-answer grading sends the question, reference answer, and student response. Tutor calls send recent conversation history (up to 20 messages) and retrieved source excerpts. Email is not needed in those prompts, but personal information placed in materials or messages may be sent. Provider-side handling is governed by that provider's terms.

Provider errors are surfaced as LLM errors; material processing returns HTTP 502. Invalid JSON gets one repair retry. Embedding failures fall back to lexical retrieval. Ollama requests use a 300-second timeout. No automatic provider failover is configured; OpenAI/Anthropic use their SDK timeout behavior rather than a project-specific timeout setting.

Estimated hosted cost for 10 active students: Not determined. “Active” usage (documents, tutor turns, generated lessons, and tokens) has no workload definition, and the app does not calculate prices. It logs token counts, operation, latency, and status in `ai_usage_logs`, but not cost. Estimate as input tokens × input rate + output tokens × output rate + embedding tokens × embedding rate, using current provider rates and a representative pilot. Ollama has no per-token API charge; hardware, power, and operations still cost money.

8. Local AI: feasibility, models, and document support

Can it run without external AI APIs? Yes. Use Ollama for both generation and embeddings; no provider API key is needed. Download model weights once using the commands above. Subsequent local inference can run without external AI APIs, provided there are no other external services in the deployment.

- The configured defaults are `llama3.1` for generation and `nomic-embed-text` for embeddings.
- Candidate model to evaluate for Arabic and English generation: Qwen2.5 7B or 14B Instruct using Q4_K_M quantization. A multilingual embedder such as BGE-M3 is another candidate. These are recommendations for evaluation, not models verified by project tests.
- PDF support extracts text using `pypdf`. There is no OCR pipeline, image upload, or vision model integration. Scanned PDFs and images require additional OCR/vision work.
- The interface includes English and Arabic localization and language-aware lesson/tutor fields. Local Arabic generation, Arabic PDF extraction, and citation quality have not been certified by acceptance testing.
- Grounding references are persisted and rendered in parts of the UI, but quality must be checked using representative materials and a real model.

Downloading images, packages, and model weights needs network access initially. There is no required third-party cloud runtime in the local Ollama configuration.

9. Local AI hardware estimates

The values below are rough planning estimates, not benchmarks or measured capacity for this application.

Use	GPU / VRAM	System RAM	CPU	Storage	Capacity / speed
Single-user testing with 7B/8B Q4	Optional; 8–12 GB if using GPU	16 GB	8 cores	30–50 GB free SSD	Single-user experimentation; latency depends on model and hardware.
Production planning for 10 simultaneous students, 14B Q4	About 48 GB VRAM as a planning target	64 GB	16+ cores	100 GB SSD	Not measured; concurrent requests may queue and throughput depends on context and serving configuration.

No response-speed target or concurrent-user guarantee can be stated without a representative load test. Benchmark latency, throughput, and queueing on the actual hardware before promising service levels.

10. Demo data and automated tests

When demo seeding is enabled on an empty Docker database:

- Student: `student@suhail.demo` / `password123`
- Administrator: `admin@suhail.demo` / `password123`
- Sample authored materials cover Biology and Algebra, including lessons, quizzes, and progress.

Use demo credentials only for local/testing environments. Set `SEED_DEMO_DATA=false` in public deployments and configure secure accounts.

- Backend, from `backend/`: `python -m pytest`
- Frontend type check: from `frontend/`, `npm run typecheck`
- Frontend production build: `npm run build`
- Focused accessibility test: `npm exec vitest -- run src/pages/__tests__/a11y.test.tsx`

Validation for this handover: 90 backend tests passed; frontend typecheck passed; the focused accessibility test passed (2 tests); production build generated the PWA manifest and service worker; Docker frontend image built and runtime smoke checks returned HTTP 200 for app shell, PWA files, icons, and `/dashboard`.

11. Requirements checklist and limitations

Requirement	Status	Evidence / outstanding work
Frontend and backend source	**Done**	Present in repository.
Dockerfiles, Compose, environment template	**Done**	Frontend image and PWA runtime smoke test passed.
Database schema	**Done**	SQLAlchemy models and startup table creation.
Schema migration scripts	**Not Done**	No Alembic or equivalent migrations.
Local Ollama support	**Done / validation partial**	Provider wired in; local model quality, language, and performance need acceptance tests.
Hosted AI providers	**Partial**	Integrations exist; production behavior and usage-based cost need validation.

Arabic and English	**Partial**	UI localization exists; local output and document support are not certified.
OCR/scanned PDFs/images	**Not Done**	Text-based PDF extraction only.
10-student concurrency benchmark	**Not Done**	Hardware suggestions are approximate; no load test.
Production deployment / live demo	**Not Done**	No live URL confirmed.
Automated tests	**Done**	90 backend tests; focused frontend checks passed.

Additional deployment concerns: password reset has no mail service. `AUTH_EXPOSE_RESET_TOKEN` defaults true in the example and should be disabled publicly. Docker dependency installation reported one high-severity npm audit finding that remains untriaged. The production build emitted a large JavaScript chunk warning.

Live demo URL: None confirmed.